

AIOPS-99: Series Summary

Series: AIOPS — Dynatrace Intelligence | **Notebook:** 8 of 8 | **Created:** May 2026 |
Last Updated: 08/27/2026

Overview

Closing notebook for the AIOPS series. Recaps the seven chapters, indexes the canonical DQL queries, lists the cross-series pointers, and surfaces the next steps for an AIOps initiative.

Audience: Anyone returning to the series after first read; anyone planning a follow-on initiative.

Table of Contents

1. [What This Series Covered](#)
 2. [Canonical DQL Queries](#)
 3. [Cross-Series Pointers](#)
 4. [Where to Go Next](#)
 5. [Source Currency](#)
-

Prerequisites

Requirement	Details
Reading order	Best read after at least AIOPS-01; selectively after the chapters you've already used

Requirement	Details
No DQL execution required	This notebook is reference material; queries are reproduced as text

1. What This Series Covered

Chapter	Subject	One-line takeaway
AIOPS-01	Overview	Three AI categories — Causal, Predictive, Generative — over Smartscape and Grail
AIOPS-02	Anomaly Detection	Five mechanisms; pick by metric character, not by habit
AIOPS-03	Problems & RCA	<code>fetch dt.davis.problems</code> ; topology completeness gates RCA quality
AIOPS-04	Davis CoPilot / Dynatrace Assist	Generative AI for DQL, summaries, explanations; not a replacement for tenant-specific knowledge
AIOPS-05	AI Models	Causal correlation, predictive baselines, generative LLM — three different model families
AIOPS-06	Integrations & Agentic	Workflow AI tasks, MCP server, AutomationEngine; agentic in suggest-mode first
AIOPS-07	Putting It Together	Detect → Investigate → Remediate; capacity, incident, regression scenarios

2. Canonical DQL Queries

Active problem feed (the most-used query in the series):

```
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| sort event.start desc
| fields display_id, event.name, event.category, event.start,
      root_cause_entity_name, affected_entity_ids
| limit 50
```

Problem category rollup — 7 days:

```
fetch dt.davis.problems, from:-7d
| summarize problem_count = count(), by:{event.category}
| sort problem_count desc
```

Problem severity rollup — 7 days: a different axis from category, and the one AIOPS-03 § 5 treats as

distinct. Note `event.severity` is typed `experimental` in the semantic dictionary — fine for triage,

not something to hard-code an integration against.

```
fetch dt.davis.problems, from:-7d
| fieldsAdd severity_label = if(event.severity == 1, "1 - critical",
                                else: if(event.severity == 2, "2 - high",
                                else: if(event.severity == 3, "3 - medium",
                                else: if(event.severity == 4, "4 - low", else: "5
- info"))))
| summarize problem_count = count(), by:{event.severity, severity_label}
| sort event.severity asc
```

MTTR by category — 30 days:

```
fetch dt.davis.problems, from:-30d
| filter event.status == "CLOSED"
| fieldsAdd duration = event.end - event.start
| summarize {
    median_mttr = percentile(duration, 50),
    p95_mttr    = percentile(duration, 95),
    problem_count = count()
},
by:{event.category}
| sort problem_count desc
```

Raw signal volume by category:

```
fetch dt.davis.events, from:-1h
| summarize signal_count = count(), by:{event.category}
| sort signal_count desc
```

Signal-to-problem compression trend:

```
// Run both, compare:
fetch dt.davis.events, from:-7d
| makeTimeseries signals = count(), interval:1d

fetch dt.davis.problems, from:-7d
| makeTimeseries problems = count(), interval:1d
```

Full reference set in [REFERENCE.md](#).

3. Cross-Series Pointers

Where AIOPS hands off:

Series	Hand-off
WFLOW	Notification routing on active problems
AUTOM	Anomaly detection settings as code (Monaco / Terraform)
ADOPT	AI Ops maturity model and adoption metrics
DASH	Problem-driven SLO and executive dashboards
K8S, DBMON, CLOUD	Domain-specific anomaly patterns layered on these mechanisms
OPLOGS, OPMIG	Pre-AI investment: clean log streams that AI can reason over

Where AIOPS is the canonical home:

- AI category model (Causal / Predictive / Generative)
- Davis problem and event data shape
- Davis analyzer family
- Dynatrace Assist surfaces and patterns
- MCP server tool catalog (problem and analyzer surface)

4. Where to Go Next

If you came here looking for a starting point, pick one based on where you are:

- **No detectors tuned yet** → AIOPS-02, then AUTOM-05/06 to put settings in code

- **Problems flooding the on-call rotation** → AIOPS-03 to understand grouping; WFLOW for notification design; ADOPT-04 for maturity framing
- **Team not using Assist productively** → AIOPS-04; pair with a working session reviewing the prompt patterns
- **Considering BYO LLM or external AI in workflows** → AIOPS-05 (model boundaries) and AIOPS-06 (integration surfaces)
- **Building an end-to-end loop** → AIOPS-07 scenarios, then operationalize with the WFLOW patterns

Three high-value next initiatives most teams should consider:

1. **Audit topology completeness.** RCA quality is bounded by Smartscape coverage. Inventory untraced services and undeclared dependencies; treat each gap as a backlog item.
2. **Move detectors to config-as-code.** Shift the Anomaly Detection app from primary configuration tool to exploration tool. AUTOM-05/06.
3. **Introduce one workflow with an AI task.** Start with the *Summarize Open Problems* pattern — high value, low risk, builds team familiarity with AI in workflows.

5. Source Currency

All notebooks in this series cite the official Dynatrace docs in [REFERENCE.md](#). Last verified: **2026-05-05**.

Areas that are evolving fast in 2026 — re-check before relying on screenshots or specific UI flows:

- Dynatrace Assist surfaces (UI placement, conversation starters, action-suggesting features)
- Davis CoPilot agentic features (preview features change naming and scope frequently)
- Workflow AI task templates (new templates ship monthly)
- MCP server tool catalog (new tools added; existing tools sometimes renamed)

When you find drift, update [REFERENCE.md](#) and bump the *Last Verified* date. If a query stops returning expected results, validate against `mcp__dynatrace__execute-dql` first — the data model is more stable than the UI.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.